2024 3rd IEEE International Conference on Artificial Intelligence for Internet of Things (AIIoT 2024)

DeepKet- Quantum Space-Efficient Word Embedding Layer for Steganalysis

Roshan Ahmed N
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
roshan.ahmed2122002@gmail.com

S. Shridevi
Center for Advanced Data Science
Vellore Institute of Technology
Chennai, India
Shridevi.s@vit.ac.in

2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT) | 979-8-3503-7212-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/AIIoT58432.2024.10574718

***Abstract*— Text-based Statistical steganography is one of the most non-human detectable methods of embedding hidden messages in plain text format which is useful in concealing information. Steganalysis is its counter, the process of detecting if a text has any encrypted data in it. This paper applies quantum computing to create DeepKet Embedding, which optimizes the space requirements for word embeddings similar to Word2Vec. DeepKet is benchmarked against existing embedding layers and a significant size reduction is achieved while maintaining accuracy for steganalysis.**

Keywords*— *Steganography, Steganalysis, Convolution Neural Network, Word2Vec, Quantum Computing, Quantum NLP, Text Classification

I. Introduction

Steganography[1] is a form of encryption that falls under concealment. The key difference is that after this encryption is done, a human viewing the data can tell clearly that there is some encrypted message in there, even if they cannot decrypt it[2]. The best example of this is in the prisoner's problem:

Alice and Bob must send a secret message to the other person, but without the 3rd party, the jailer, finding out that a message has been encrypted. If standard encoding methods are applied, the carrier text will be visibly distinct from other texts, which will alert the jailer that there was an attempt to exchange sensitive information. However, steganography embeds a message within the carrier text in a way that a person who does not know the method used, in this case, the jailer, will be unable to tell that any embedded message was there.

An example of this is writing specific letters in capital or a different font, that may spell out a code word or a sentence that carries sensitive information[3].

Modern steganographic techniques can change sentences into grammatically different ones to hide codes within them [4]. Most people cannot perceive such changes as being an embedded code, and decrypting it takes just as long for someone unaware of the algorithms. This is why steganalysis, or the process of detecting steganographic texts from normal texts, is crucial to decrypting messages to not waste time or computational power.

One way to detect linguistic embedding is by using the word cosine correlation, which may indicate that one of the sentences in the carrier text may not be human-written, and could have been passed through an embedding algorithm. Proposed models[5] use convolutional neural networks (CNNs) to measure cosine correlation and compare all the extracted features to check for anomalies. Additionally, Recursive neural networks (RNNs) have also been shown to have good accuracy in these use cases.

Quantum computing methods may help to optimize this process. However, to run language models on current space-constrained quantum systems, smaller models that maintain efficiency are required for it to be feasible. Thus there is a motivation to find quantum-friendly methods that can be deployed in space-constrained systems[6].

DeepKet is a text embedding algorithm (similar to Word2Vec) that is used in natural language applications. Standard embedding algorithms are highly space-consuming. DeepKet, the proposed model of this paper, aims at reducing the space consumed by utilizing quantum computing models by using Kets to represent embedding vectors while maintaining the accuracy seen in classical algorithms.

This proof-of-concept project benchmarks DeepKet against Word2Vec and Word2Ket [7]. The benchmark model will detect steganographic texts in groups of normal texts to minimize space complexity and maintain the accuracy of classical models.

II. Literature Review

Encryption of data can be easily detected as encrypted data looks significantly distinct from normal data, and cannot bypass a human checking it. Thus, steganographic methods became popular. One of the most common steganographic techniques, image steganography, requires a communication medium that supports image transfer without compression, which limits the flexibility of image steganography.

Text steganography has 3 subcategories. Format steganography is created by capitalizing certain letters, or keeping certain letters in a different size or font. Format steganography breaks if there is a change in format, which happens commonly on the internet. Linguistic steganography is using wordplay, synonyms, or riddles to make a certain text have a hidden meaning. The overall data per sentence varies a lot based on context and there is no guarantee that the receiver can decrypt the wordplay given.

Another variant of text steganography is ASCII-armored data, which manipulates and re-calculates ASCII values. This however is limited as encrypted ASCII text does not form coherent words and can easily be intercepted by a human check

The last one is statistical steganography. By measuring the probability of a certain word occurring in certain orders, if both sides have the same statistics database, the sentence can be reverse computed to get the original probabilities

979-8-3503-7212-0/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:09:06 UTC from IEEE Xplore. Restrictions apply.

which can contain binary data. An example of this is executed using generative Markov chains [8].

Markov chains are high-entropy encryption machines. The execution of these chains happens by reading one word, identifying what the next word can potentially be, and choosing from one of them. For example, the word “blue” followed by “sky” could mean 0, while “blue” followed by “sea” could mean 1.

Some variants of Markov chain steganography use Huffman encoding [9] to increase the amount of potential data transfer and variation in word combinations. For example, in the same situation with Huffman encoding applied, “blue sky” is 0, “blue sea” is 10, and “blue car” is 11.

A single unit of encryption can have more than one word. Using a single word per data token is unigram encryption while 2 words per data token are called bigram encryption [9]. The higher the encryption is, the longer sentences have to be to encrypt the same amount of data, with the payoff that the sentences become more grammatically coherent

Steganalysis is the counter for steganography. Its purpose is the detect messages that have steganographic content in them. Steganalysis for semantic steganography has been achieved using Convolutional neural networks [10] but recursive neural networks [11] have also been shown to achieve good results.

The key proposal of this paper is the embedding layer, a neural network layer that holds word embeddings. Word embeddings are natural language processing tools that allow converting a single word into feature vectors to give more insight into the correlation between words. This method of creating word embeddings is called Word2Vec.

A variant of word embeddings can be achieved using compositional code [12]. Computing word embeddings in runtime by using multiple smaller vectors can lower the space taken by the embedding layer. A similar concept is applied when creating encryption keys using smaller encryption keys. The integrity of the keys is maintained using the Kronecker product.

When vectors are replaced with quantum states, it is called a ket. When considering Kets as a potential computation element, more options for embedding layers open up. Word2Ket uses Kets with multiple inner products to recreate the embedding matrix.

A collection of key papers and their defining concept can be found in Table I.

TABLE I. PAPER-WISE KEY CONTRIBUTION

Reference No	Key Contribution
[3]	Theory about steganography
[7]	Word2Ket algorithm that is used for benchmarking
[9]	Markov chain steganography implementation that is used to generate the data set
[10]	CNNs sucessfully used for Steganalysis
[11]	RNNs can be used for steganalysis as well

III. THEORY

A. Steganography

Semantic steganography can be implemented by using an entropically secure cipher. If there is a cipher has a high level of entropy, it can be used to encrypt plain text.

Semantic steganography uses semantic redundancies to encrypt data in text. Some intentional semantic errors can be placed at calculated positions that can be reverse-engineered to extract the original data. However, the number of combinations of potential ciphers is high, making plain text semantic encryption an entropically secure cipher.

Markov chains can be used to create an entropically secure cipher. The goal of an entropically secure cipher is to make it hard to decode if the original cipher, in this case, the Markov chain, is unknown. Given a large number of states, Markov chains become harder to reverse engineer. When hidden among a group of unencrypted texts, the overall entropy becomes even higher.

Markov chain encrypting is a way of writing sentences based on a certain hidden binary stream. The machine looks at the root word and attempts to find the word that most likely follows the root word, and assigns a certain binary value to them. For example, the root word “blue” may be followed by “sea” or “sky”. The machine looks at the bit to be encoded and chooses the next most followed word. For example, 0 may indicate “sea” and 1 indicates “sky”. The next word is picked as the root word and the process continues forming an encrypted message.

The intended target with the original chain will be able to decrypt it by tracking each root word and seeing what the next word implies in terms of the bitstream. Bigram encoding is similar, but it uses 2 or more words instead of a single word to make a more coherent-looking sentence.

Table II gives an example of how a word is encrypted in unigram and bigram encryption.

The need to distinguish between encrypted and unencrypted messages becomes crucial to lowering the entropy of the secret message for the interceptor to decrypt the right message. Additionally, picking out the encrypted message lowers the computational effort, where the interceptor only has to decrypt an encrypted message instead of all incoming messages. This is why steganalysis, the method to find out if a message is an output of steganography, is necessary.

B. Word2Ket

Word2Ket is a variation of the classic Word2Vec word embedding model that lowers the spatial complexity of word

TABLE II. MARKOV ENCRYPTION OF SECRET MESSAGE.

Category	Text
Secret Message	“Secret”
Unigram Encryption	“How glad to Bilibin's sleeve and crystal glitter dazzled by such a hair's breadth of the house”
Bigram Encryption	“How can we ever conceive either complete freedom of all among the opinions and wishes among those who make any plans. Pierre finished his tale about Mademoiselle Bourienne. This quivering”

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:09:06 UTC from IEEE Xplore. Restrictions apply.

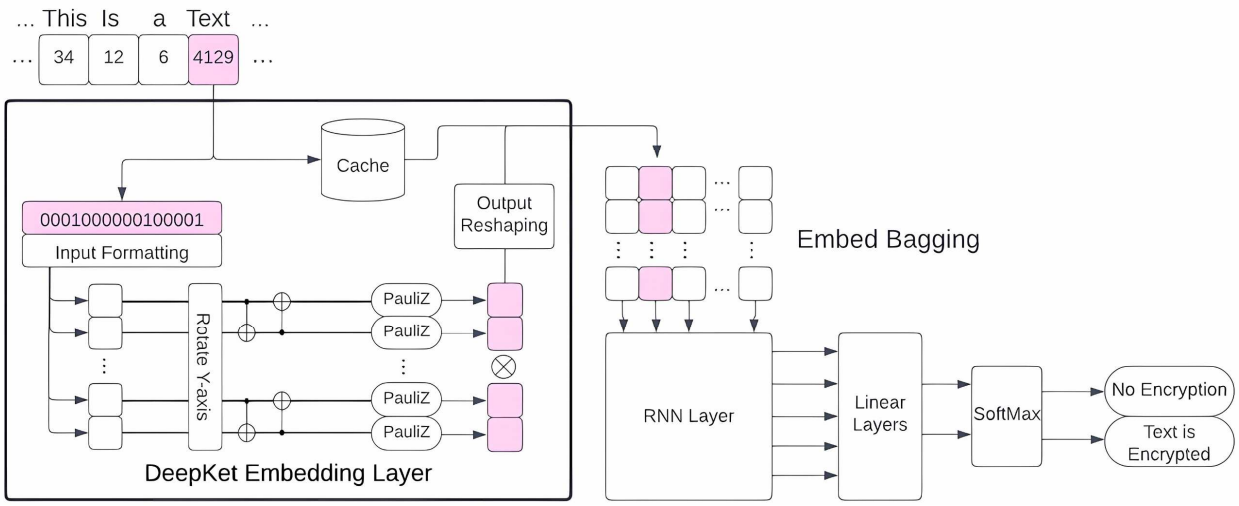


Fig. 1. Architecture of steganalysis model and DeepKet embedding layer

embeddings. Word2Ket utilizes 2 key features that help it achieve this goal:

- Storing the embedding matrix as lower-dimensional tensors called Kets.
- Performing a tree computation during runtime to retrieve the vector for the specific index value.

The paper referenced claims to be quantum inspired, by stating that the mechanics behind Kets is similar to how entangled tensors can be used to represent any vector in the Hilbert space, a higher dimensional space that all quantum states exist in. However, the Word2Ket paper only represents all entangled tensors using classical tensor products without any quantum aspect inherent in them. This logic can be extended into the quantum domain and find a way to implement this.

C. *Introducing DeepKet.*

DeepKet is the proposed model in this paper. **DeepKe**t uses a series of quantum functions on a variational quantum circuit to create a hybrid classical-quantum-classical embedding layer. The supporting theory, assumptions, and equations are as follows:

All quantum functions can be represented as a measurement of a series of quantum transformations. These series of transformations output the Hilbert Space, represented in the form of a matrix, which holds all quantum states.

$$\langle \psi | \otimes | \phi \rangle \quad (1)$$
$$\langle \psi | \phi \rangle \quad (2)$$

Here, $|\phi\rangle$ is a placeholder for a quantum state called ket, and $\langle \psi|$ is a vector with possibilities of each quantum state called bra. By taking the inner product [13] between them we get the Hilbert space matrix, which is denoted in the bracket notation.

The goal is to hold all embedding values within the Hilbert space, such that it can be accessed using a single measurement. Measurements can be represented by the general equation (3)

$$|\psi\rangle\langle\psi| \otimes |\phi\rangle \quad (3)$$

Here $|\psi\rangle\langle\psi|$ represents the measurement axis. This type of product gives a single-element output which is the measurement needed. In this case, ψ could represent the binary index number that we want to measure e.g. $|0101\rangle$ for index number 5.

Grouping this equation differently, it is similar in structure to the definition of a Hilbert space.

$$|\psi\rangle\langle\psi|\phi\rangle \quad (4)$$
$$|\psi\rangle \otimes \langle\psi|\phi\rangle \quad (5)$$

What this means is that any ket can be selected from the Hilbert space by temporarily creating said Hilbert space by tweaking the entangled values and using $|\psi\rangle$ as the input. In the classic version of Word2Ket, this is created by using lazy tensors to compute the whole Hilbert space using the Kronecker product (Inner tensor product) and then selecting the necessary column from them.

In the quantum Word2Ket implementation, the Hilbert space does not need to be manually computed, as it is an inherent byproduct of creating any combination of qubits. To read the required embedding tensor from the Hilbert space, once the initial state is created using an input tensor, then apply the quantum gates that create the desired Hilbert space.

The motivation to use Quantum computing comes from the appeal of being able to store higher dimensional data in lower dimensional data structures[14] which are perfect for optimizing feature vectors

IV. Methodology

A. *Variational Quantum Circuit*

DeepKet is implemented by making a variational quantum circuit [15] using Pennylane.

Pennylane is an open-source framework that is used for quantum computation. One of the useful aspects of PennyLane is its low-level manipulation of Qubits that

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:09:06 UTC from IEEE Xplore. Restrictions apply.

allows running sequences of actions on qubits that can then be measured for some output. This type of circuit is called a variational quantum circuit.

Each quantum circuit represents a circuit that has a certain number of wires, represented by the variable n_qubits. According to the Bloch sphere representation, there are 3 axes on which qubits can be manipulated.

The input value rotates the qubit on the X-axis. The next layers, Entangler layers, rotate a qubit on the Y axis and then apply a circular CNOT gate which allows adjacent qubits to influence each other and is the core of the quantum machine learning layer. The final layer is the measurement layer that uses a PauliZ gate to measure the expected value against the Z axis.

Using a Python decorator, Pennylane can identify this circuit as a quantum node which can then be converted into a PyTorch layer [16].

B. *Model Architecture*

The model must be able to read a text and determine if there is a steganographic message hidden in it, or if it is just a normal text. It takes in a paragraph of a maximum of 100 words as an input and outputs the SoftMax probability determining if it is a steganographic text or a normal text.

The steganalysis model used for benchmarking is based on a Recursive neural network (RNN) many-to-one model. The assumption behind this is that there may occur words in short distances that should not contextually be in those places. RNNs are capable of extracting such anomalies that we can safely use to detect if there is encryption in that message.

Each word in the sentence will first be converted to an integer using one hot encoding called a vocabulary. This vocabulary is generated by looking at the words that occur in a reference text.

The embedding layer is the main focus of the benchmark model as 2 major variants are being tested here. Word2ket and Word2Ket XS which both use slightly different ways to calculate the embedding matrix during runtime.

The embedding matrix can be calculated by a series of quantum functions. **The model DeepKet** uses this to generate an output vector, taking in the index in binary form as an input. This significantly lowers the number of trainable parameters in the embedding layer. Fig 1 has a visual representation of the whole architecture and Table III describes the layer hyperparameters

TABLE III. MODEL DESCRIPTION (DEEPKET LAYERS ARE IN BOLD)

Layer	Attributes
Input formatting	Linear, 16 inputs, qubits x 2 output
Quantum Circuit	2 parallel Quantum circuits. X-axis angle embedding,, Y-axis entanglement
Inner product Output	Linear, qubits x 2 input, embedding size (64) output
RNN layer	64 input, 32 output, 3 hidden layers
Linear	32 input, 32 hidden, 2 output, ReLU activation
Output	Softmax, 2 outputs : normal text(0) or steganographic text(1)

C. ***Introducing DeepKet***

Deepket is executed in this benchmark experiment using Pennylane and PyTorch. PyTorch has a feature where you can use Pennylane quantum nodes as layers. A hybrid model with classic and quantum layers will be used to create a new embedding layer. To do this, the required formatting of the PyTorch Embedding layer must be maintained.

The key features of the embedding layer [17] are as such:

- It must function as a look-up table, single input must give a full tensor as an output
- For batches and sentences, the layer must be run multiple times and the outputs for each must be collected and sent as a 3d layer.
- Embedding layer outputs are normalized [18] along the index value

To recreate the following embedding layer, a nested for loop runs over the batch count and sentence length to get the embedding tensor individually. Each tensor is then passed through a normalize function to get the normalized embedding tensor.

We create a 2d list to hold all words for the first sentence, this is called bagging. The inner loop bags all the words first to create a sentence bag, then loops over all sentences in the batch to create a batch bag, which is then returned as a 3d tensor of size Embedding Size x length x batch size.

D. *Dataset generation*

Most steganography datasets are in Mandarin as the amount of information per symbol is significantly higher. For this proof-of-concept project, an English book “The Black Swan: The Impact of the Highly Improbable” [19] by Nassim Nicholas Taleb is used as the generation root. The Markov chain steganographer reads the book and makes decisions based on it.

Sentences from the book are extracted and used for Markov chain unigram and bigram encoding. A large chapter has been encrypted in both unigram and bigram, and is truncated at 100 words per data point. The same is done to regular content from the book, sampled in Table IV.

TABLE IV. SAMPLE DATA

Category	Text
No Encryption	Late, late, so late! and dark the night and chill! Late, late, so late! but we can enter still. Too late, too late! ye cannot enter now. No light had we: for that we do repent; And learning this, the bridegroom will relent. Too late, too late! ye cannot enter now. No light: so late! and dark and chill the night! O, let us in, that we may find the light!
Unigram Encryption	Since I would to banjo playing pirate ships go My ear. In sweet wine spill grains of distraction's indirection. Scanning all posterity Even beneath your picture a body s a darling you keep my love of the floating behind her fierceness when rocks occasionally break. And stronger by. They parted.
Bigram Encryption	I looked the other. One drop fell from power lines. As his strength slay thy worm in Hell. And he knew. What will succeed it now. We ask that imagination discover the path to the storm cellar on a tree. And leaves I beat flex crouch leap. The con of it. Drowned tripper fat crawls into hollow cuffs to be. Maybe it says nothing. Then I know not where.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:09:06 UTC from IEEE Xplore. Restrictions apply.

25% of the dataset is unigram encrypted, 25% is bigram encrypted and 50% is non-encrypted text from the book. The goal is to distinguish between encrypted and non-encrypted texts. thus, an equal count of both classes of texts is optimal.

V. Implementation and Inferences

A. DeepKet Model

DeepKet is implemented by creating a hybrid layer that has 3 sublayers. The 3 sublayers are as follows:

- Input Formatting/Resizing layer
- Quantum Circuit
- Inner product layer

Both resizing and inner product layers are implemented using linear layers while the quantum circuit is a split layer with 2 parallel circuits [20]. The input and outputs for the quantum circuit layer are twice the number of qubits.

The index value is taken and converted to binary, which is then passed into the formatting layer. Binary directly cannot be used for angle embedding, so a formatting layer is needed to convert binary into non-perpendicular input angles. Rotation and entangler layers are handled by the layer weights under PyTorch and do not need formatting.

The inner product layer computes the Kronecker product of the 2 concatenated output Kets. Due to the high level of information stored when using qubits, this gives a decent embedding output that is unique for adjacent indices.

The resizing of the input and output layers allows for freedom in choosing the embedding dimensions and does not restrict it strictly to powers of 2.

B. Memoization

Running quantum circuits for each index is a very time-consuming aspect due to the number of times it needs to run. Frequently used indices e.g. words for unknown, pronouns, and articles can be cached in a dictionary for temporary use during each batch run. [21]

The model first checks the cache for the required index, and if available, directly returns it. If not, the model runs normally and caches the embedding tensor at the end of it. This however can only run once per batch and must be cleared at the end of the batch to avoid bloating.

C. Training

Transfer learning is used here to speed up the whole training process.

An initial model with a standard embedding layer is trained to maximize output accuracy and is used as a benchmark. The maximum output accuracy obtained was 92.7% which is the goal accuracy of the models for perfect replacement. The comparison of models is shown in Table V.

After this, the parameters are frozen and the embedding layer is replaced with multiple models to compare and benchmark.

Word2KetXS which was the classical version of this paper's proposed model is also benchmarked. DeepKet brings better accuracy than Word2Ket but has a larger size than it, putting it in between both options

As the results in Table V show, in creating a significant reduction in embedding layer parameter sizes, the overall accuracy has a low drop, allowing the usage of embedding size reduction only when the value of size reduction is more than the accuracy lost in the process. Best quantum outputs are shown in Table VI.

The original model was optimized using stochastic gradient descent and the transfer embedding layers were trained using Adam optimizer.

D. Inference:

When using binary directly into the variational quantum circuit, the output was unchanged and was always zero. This was why the input formatting layer was added. It is an unfixed bug in Pennylane which was bypassed using this fix.

The steganalysis model fails for small texts less than 20-30 words and works best for texts more than 50 words.

Non-normalized and direct output from the embedding layer was lowering the average accuracy of the models, thus the normalization function was added.

High batch size overflows the memo cache. This can be fixed by setting a max limit for the memo cache, or by running an insert sort with frequency in the memo cache with a cutoff size. For lower complexity, in this model, the batch size itself was reduced.

Having too low of a batch size made the memo cache redundant, so after testing with multiple batch sizes, a batch size of 8 gave the fastest training time without significant overflows.

Larger models such as 8 qubits or more require longer training time and thus do not perform just as well in the 150-batch cutoff benchmark. Other than such an anomaly, we can see that the closer the qubit count goes to the expected ket size, the better the accuracy is.

Lastly, the existence of the normalization layer increased the accuracy of the overall model, and thus it needed to be included. The model size has been significantly reduced while maintaining similar accuracy, and the best model has been chosen

TABLE V. Embedding Layers Comparison

Embedding Layer	**Accuracy comparison**				
	Wires	***Layers***	***Normalize***	***Accuracy***	***Parameter count***
PyTorch default classical embedding layer	n/a	n/a	n/a	**92.7%**	1,194,112
DeepKet (proposed)	2	1	Yes	**70.8%**	392
			No	**60.4%**	392
		2	Yes	**68.6%**	720
	4	1	Yes	**85.4%**	720
		2	Yes	**83.33%**	720
	8	1	Yes	**83.33%**	1376
Word2Ket[7]	Order 4 , Rank 3			**81.2%**	288
XS[7]				**72.7%**	252

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:09:06 UTC from IEEE Xplore. Restrictions apply.

TABLE VI. SAMPLE RESULTS

Test Case	Label	Predicted
I thank you, kind and best beloved friend, With the same thanks one murmurs to a sister, When, for some gentle favor, he hath kissed her, Less for the gifts than for the love you send, Less for the flowers, than what the flowers convey; If I, indeed, divine their meaning truly, And not unto myself ascribe, unduly,Things which you neither meant nor wished to say, Oh! tell me, is the hope then all misplaced? And am I flattered by my own affection? But in your beauteous gift, methought I traced Something above a short-lived predilection, And which, for that I know no dearer name, I designate as love, without love's flame.	False	False
you can t see how the chief Of joys seems not to crack. But all you meet and mingle. When the mainsheet starts the kind of petition. No dear that seems. The war. Of the dayin which I see A child. How have you. What god was here now. Snows Order their populations forth. And the crying. When the lover absent is. What who do you need four with four the progression begins. And look up and steal the signet ring of its counterpart. Beyond the dikes. No one steals. Children fell at Marathon. The sculptures grow by the city angled to abstraction. Starting up and walk from room to room Between attic and basement. Coffee would be called hoses. And there is something urgent here. But our children	True (Unigram)	True
Bet you have dappled the downs and valleys at this late date. And you were once employed as one dead by light too bright. They rattle reason out through sliced gills. The truth so let the tongue. But I Have of fading out still there are times when this was his knuckle that made my own cupped hands. I picked out a worm to place. The lungs and the wild unrest that lives in Ájyadi. I breast yon deep and fair. Remember me when I come from. So let my heart. Out on a beach. And he always tells himself I just can't catch. The lungs and pulse. What matter the rush on paperAnd open as though the girl from Scarborough liked being slappedDown the hall nor had the sense of what	True (Bigram)	True

E. Future work

The main issue with this experiment lies in the lack of computational resources and scalability. For larger number of wires or qubits, the amount of time taken for training increases drastically. The application of DeepKet in larger models needs to be tested and implemented. The quantum version of the embedding layer took 5 epochs to reach the optimal accuracy. With better computational resources, further experimentation can be done.

REFERENCES

[1] Johnson, Neil F., and Sushil Jajodia. "Exploring steganography: Seeing the unseen." Computer 31.2 (1998): 26-34.

[2] Russell, Alexander, and Hong Wang. "How to fool an unbounded adversary with a short key." Advances in Cryptology—EUROCRYPT 2002: International Conference on the Theory and Applications of Cryptographic Techniques Amsterdam, The Netherlands, April 28–May 2, 2002 Proceedings 21. Springer Berlin Heidelberg, 2002.

[3] Majeed, Mohammed Abdul, et al. "A review on text steganography techniques." Mathematics 9.21 (2021): 2829.

[4] Mansor, Shaifizat, Roshidi Din, and Azman Samsudin. "Analysis of natural language steganography." International journal of computer science and security (IJCSS) 3.2 (2009): 113-125.

[5] Yang, Zhongliang, et al. "TS-CNN: Text steganalysis from semantic space based on convolutional neural network." arXiv preprint arXiv:1810.08136 (2018).

[6] A. Alexander and D. Widdows, "Quantum Text Encoding for Classification Tasks," in 2022 IEEE/ACM 7th Symposium on Edge Computing (SEC), Seattle, WA, USA, 2022 pp. 355-361

[7] Panahi, Aliakbar, Seyran Saeedi, and Tom Arodz. "word2ket: Space-efficient word embeddings inspired by quantum entanglement." arXiv preprint arXiv:1911.04975 (2019).

[8] Moraldo, H. Hernan. "An approach for text steganography based on Markov chains." arXiv preprint arXiv:1409.0915 (2014).

[9] Luo, Yubo, et al. "Text steganography based on ci-poetry generation using Markov chain model." KSII Transactions on Internet and Information Systems (TIIS) 10.9 (2016): 4568-458

[10] Yin, Wenpeng, et al. "Comparative study of CNN and RNN for natural language processing." arXiv preprint arXiv:1702.01923 (2017).

[11] Oyelakin, Bamgboye Pelumi, et al. "Text Classification Using Recurrent Neural Network and Support Vector Machine on a Customer Review Dataset." Journal of Theoretical and Applied Information Technology100 (2022).

[12] Shu, Raphael, and Hideki Nakayama. "Compressing word embeddings via deep compositional code learning." arXiv preprint arXiv:1711.01068 (2017).

[13] Karlekar, Nandkishor P., and N. Gomathi. "Kronecker product and bat algorithm-based coefficient generation for privacy protection on cloud." International Journal of Modeling, Simulation, and Scientific Computing 8.03 (2017): 1750021.

[14] Subbiah, Geetha, et al. "Quantum transfer learning for image classification." TELKOMNIKA (Telecommunication Computing Electronics and Control) 21.1 (2023): 113-122.

[15] Haghshenas, Reza, et al. "Variational power of quantum circuit tensor networks." Physical Review X 12.1 (2022): 011047.

[16] Baronia, Dhruv. "Hybrid Quantum-Classical Neural Networks for Text Classification." Authorea Preprints (2023).

[17] Rothman, Denis. Transformers for Natural Language Processing: Build innovative deep neural network architectures for NLP with Python, PyTorch, TensorFlow, BERT, RoBERTa, and more. Packt Publishing Ltd, 2021

[18] Garbin, Christian, Xingquan Zhu, and Oge Marques. "Dropout vs. batch normalization: an empirical study of their impact to deep learning." Multimedia tools and applications 79.19 (2020): 12777-12815

[19] Nicholas, Nassim. "The black swan: the impact of the highly improbable." Journal of the Management Training Institut 36.3 (2008): 56.

[20] Hrinchuk, Oleksii, et al. "Tensorized embedding layers for efficient model compression." arXiv preprint arXiv:1901.10787 (2019).

[21] Della Toffola, Luca, Michael Pradel, and Thomas R. Gross. "Performance problems you can fix: A dynamic analysis of memoization opportunities." ACM SIGPLAN Notices 50.10 (2015): 607-622.

Authorized licensed use limited to: Universita degli Studi di Salerno. Downloaded on July 02,2026 at 10:09:06 UTC from IEEE Xplore. Restrictions apply.